A systematic mapping study on the research landscape of LLM-based code clone detection

Bowen Jiang[a], Mitchell Ruffolo[a], Aaditya Kulkarni[a], Teia Canini[a], Eman Abdullah AlOmar[a,*]

[a]*Stevens Institute of Technology, Hoboken, NJ, USA*

Abstract

Context: Code clone refactoring is the practice of identifying and manipulating code duplicates to enhance code quality. Code duplication can pose many difficulties in programming, making code clone refactoring a prominent topic of research. Initial models of code clone detection were introduced using textual analysis in NiCad. Soon after using token-based models, we are now using models using large language models (LLMs). LLMs are AI models trained on large-scale datasets, enabling them to understand programming problems at a high level. LLMs such as GPT, Llama, Starcoder, and Falcon have led new code clone detection ambitions. Because of the complexity and difficulty of many coding problems, LLMs are able to comprehend them and create results at an extremely high level. This has led to the usage of LLMs such as Claude, ChatGPT, and Gemini.

Objectives: The aim of our research is to provide a comprehensive analysis of existing literature on LLM-based code clone detection practices. Our research will provide a comparison between primary research studies, giving us an overall evaluation of the landscape of clone detection, with the rise of LLM models.

Methods: Our research focused on 28 primary studies published within the past two years. These studies were selected through a combination of keyword-based database searches targeting terms related to LLMs, code clone detection, and refactoring. Human validation was used to confirm that each study directly addressed the core themes of LLM-driven software engineering practices.

Results: Our analysis of 28 studies shows that LLMs improve code clone detection, particularly for more complex clones such as Type-3 and Type-4. ChatGPT-4 outperformed earlier versions across multiple benchmarks. Few-shot and context-specific prompting were the most common and effective techniques, appearing in 65% of the studies and contributing to higher precision and recall. However, prompt sensitivity remains a major issue because subtle wording changes can lead to substantial variation in LLM responses. Most studies

*Corresponding author

Email addresses: bjiang14@stevens.edu (Bowen Jiang), mruffolo1@stevens.edu (Mitchell Ruffolo), akulkarn3@stevens.edu (Aaditya Kulkarni), tcanini@stevens.edu (Teia Canini), ealomar@stevens.edu (Eman Abdullah AlOmar)

were limited in the programming languages tested, as Java was used in 83% of the studies and Python in 50%. Type-4 clone detection remains a challenge, with LLMs having difficulty.

Conclusion: Future research on code clone detection and refactoring requires specific research directions. Specifically, multilingual testing, prompt sensitivity, and generalization. As research progresses, examining these methods will enable the effective, repeated use of LLM models in coding problems. Existing studies focus on detection rather than downstream refactoring applications, highlighting a gap in actionable tooling. Addressing these challenges will be essential for deploying LLMs in reliable, repeatable software engineering workflows.

Keywords: code clone, quality, llms, literature review.

---

1. Introduction

Software engineering systems are rapidly expanding in size and complexity, making it increasingly challenging for developers to maintain code quality and reduce technical debt efficiently. Code clone detection and refactoring play a key role in enhancing maintainability and minimizing bugs. When performed manually, these tasks can consume considerable time due to their complexity and scale. There have been efforts to automate these processes using traditional tools such as NiCad [1] and CCFinder [2] for clone detection and JDeodorant [3, 4] and AntiCopyPaster [5–8] for refactoring. Although these clone detection tools effectively identify syntactic clones (Types 1, 2, and 3), they often struggle with semantic clones (Type 4) and require substantial manual setup. Recent advancements in large language models (LLMs) such as Codex and ChatGPT provide a platform for developers to build better tools to automate software engineering tasks [9]. Because previous tools struggled with more complex tasks, LLMs can enable rapid tool evolution, enabling them to solve and perform increasingly complex problems.

The software engineering tasks of code generation and summarization have been successfully demonstrated by LLMs such as Codex and ChatGPT. The two essential areas in which automation can reduce technical debt and enhance maintainability are clone detection and refactoring. The semantic complexity of source code makes LLMs an innovative method to discover patterns which leads to meaningful code suggestions.

Code clones can be categorized into four types based on their similarity characteristics [10]. **Type-1 clones** are exact copies of code fragments, differing only in whitespace, comments, and layout. **Type-2 clones** are syntactically identical fragments that differ in variable names, literal values, and types. **Type-3 clones** are fragments where statements have been added, removed, or modified. Finally, **Type-4 clones** are semantically equivalent fragments that implement the same functionality but with different syntax. Listings 1–

4 illustrate these four clone types with concrete examples[1]. Code clones are typically managed via refactoring, which involves restructuring existing code without altering its external behavior [11, 12]. This process can remove clones by extracting shared code into reusable methods or classes, enhancing maintainability and minimizing duplication [13]. Listing 5 demonstrates a typical refactoring case where a method with multiple responsibilities is broken down into single-purpose methods.

The rapid expansion of LLM applications in development tools has led to an increasing number of studies assessing their ability to perform automated code clone identification and refactoring. Our search identified 28 primary studies on this topic, each with different evaluation metrics. These metrics included the tools used, LLMs used, datasets used, and languages evaluated. The research investigates how prompt design, together with training data and model architecture, influences the quality of generated results. The rapid development of LLMs, together with the diversity of application tasks, poses challenges for authors seeking to compare existing findings, as findings remain scattered across studies. The existing literature surveys related to code clone detection and refactoring primarily examine traditional methods and pre-LLM approaches, but they do not consider the current applications of modern LLMs such as GPT-4, and CodeBERT. The systematic literature reviews (SLRs) establish a solid base regarding clone taxonomy and detection tools and refactoring techniques yet they neglect to assess or evaluate the fast developments in LLM-based solutions. The recent LLMs receive no evaluation regarding their performance on semantic clones, their prompt engineering capabilities, or their generalization limitations across languages that impact tool reliability. To address the above limitations, this paper aims to develop a knowledge base on LLMs for code clone detection. As software engineering rapidly evolves, organizing this fragmented research is essential to help authors identify best practices, avoid redundant efforts, and accelerate progress in LLM-based clone detection.

```
// Fragment 1: Calculates the sum of an array
int total = 0;
for (int i = 0; i < array.length; i++) {
    total = total + array[i];
}
return total;

// Fragment 2: An exact copy, ignoring comments and whitespace
int total = 0;
for (int i = 0; i < array.length; i++) {
    total = total + array[i];
}
return total;
```

Listing 1: Type-1 Clone: Exact copies differing only in whitespace and comments.

```
// Fragment 1: Calculates the sum of an array
int total = 0;
for (int i = 0; i < array.length; i++) {
```

[1]Examples are generated using Manus.

```
    total = total + array[i];
}
return total;

// Fragment 2: Same structure, but variable names are different
int result = 0;
for (int j = 0; j < data.length; j++) {
    result = result + data[j];
}
return result;
```

Listing 2: Type-2 Clone: Syntactically identical but with renamed identifiers.

```
// Fragment 1: Calculates the sum of an array
int total = 0;
for (int i = 0; i < array.length; i++) {
    total = total + array[i];
}
return total;

// Fragment 2: Similar logic, but a statement is added (logging)
int total = 0;
for (int i = 0; i < array.length; i++) {
    System.out.println("Adding: " + array[i]); // Added statement
    total = total + array[i];
}
return total;
```

Listing 3: Type-3 Clone: Similar code with statements added or removed.

```
// Fragment 1: Calculates the sum of an array using a for loop
int total = 0;
for (int i = 0; i < array.length; i++) {
    total = total + array[i];
}
return total;

// Fragment 2: Same functionality, but implemented with a stream
return Arrays.stream(array).sum();
```

Listing 4: Type-4 Clone: Semantically equivalent but syntactically different implementations.

When defining our research questions (RQs), we framed them around the quantitative and qualitative aspects of the papers as follows:

- **RQ_1: What approaches were considered by the primary studies (PSs) for LLM-driven code clone?** We pose this question to categorize and study the effects of the existing approaches for LLM-driven code clone.
- **RQ_2: What datasets and benchmarks have been used to evaluate LLMs for code clone tasks?** We posed this question to examine the various datasets and benchmarks used to evaluate LLMs for code clone and further understand the design of the experiments in each study.
- **RQ_3: What prompt techniques have been used to perform code clone tasks?** We posed this question to examine what prompting methods authors have been exploring. By employing the best prompting strategies, we are able to help reduce hallucinations and maximize efficiency.

```java
// BEFORE REFACTORING
public void calculateCircleMetrics(double radius) {
    double area = 3.14 * radius * radius;
    System.out.println("Area: " + area);

    double perimeter = 2 * 3.14 * radius;
    System.out.println("Perimeter: " + perimeter);
}

// AFTER REFACTORING
public double calculateArea(double radius) {
    return 3.14 * radius * radius;
}

public double calculatePerimeter(double radius) {
    return 2 * 3.14 * radius;
}

public void printCircleMetrics(double radius) {
    System.out.println("Area: " + calculateArea(radius));
    System.out.println("Perimeter: " + calculatePerimeter(radius));
}
```

Listing 5: Refactoring Example: Extract Method to eliminate code duplication.

- **RQ_4: How accurate are LLM-generated refactoring suggestions?** This RQ investigates the accuracy of refactoring suggestions generated by LLMs. To this end, we perform quantitative, qualitative, comparative, and correctness data analysis of the LLM-based tools and approaches for each study.

- **RQ_5: What are the key challenges in automating clone detection and refactoring using LLMs?** We pose this question to identify and understand the difficulties that arise with LLM-based code clone and analyze possible methods to address them.

The contributions of this paper are summarized as follows:

- A comprehensive analysis of existing literature on LLM-based code clone across prompt engineering techniques, tool supports, and clone types.

- A breakdown of LLM-driven clone approaches and an outline of strengths and applications of major refactoring methods.

- Identifying open issues in existing research and recommendations for future research directions, including concerns, prompt engineering strategies, tools, and refactoring methods implementations.

The remainder of the paper is structured as follows: Section 2, a review of existing studies related to LLM-driven code clone. Section 3, an outline of our study setup in terms of search strategy and study selections. Section 4, the findings of our research and analysis. Section 5, concerns and call-to-action for possible future research. Section 6, the threats to validity of our work, and concluded with Section 7.

2. Related Work

Lei *et al.* [14] reviewed current available knowledge on the application of deep learning (DL) in the detection of code clones. It included a comprehensive description of DL, code clones and their types, datasets used, neural networks and their underlying techniques, embedding techniques in general and about their popularity. The authors analyze the publicly available deep learning-based clone detection tools developed by the works studied in the paper. This piece of literature serves as a good summary of the performance of different DL techniques and DL-based clone detection tools that are publicly available. A survey of software clone detection techniques by Sheneamer and Kalita [15] discussed types of code clones and various phases of the code detection process, such as pre-processing, transformation, match detection, formatting, filtering, and aggregation. It also discusses different styles of clone detection, such as text-based, token-based, tree-based, metric-based, graph-based, and hybrid. Metrics such as precision, recall, scalability, and portability are used for evaluation. The evaluations of this paper also suggest that no single approach excels in all metrics and that, despite the modern improvements in technology, the Type-3 and Type-4 clones still remain challenging. Rattan *et al.* [16] conducted a systematic review of software clone detection, analyzing 213 studies selected from a pool of 2039 articles across leading journals and conferences. The review aims to classify existing research, evaluate detection techniques and tools, and highlight gaps and future directions. The review identifies 13 intermediate code representations and 24 match detection algorithms. More than 60 tools are reviewed, with token-based tools such as CCFinder and text-based tools such as Duploc widely used. Tree-based tools (e.g., CloneDR) offer high precision but lower recall, while token-based tools have high recall but lower precision. Roy and Cordy [10] provided an extensive survey of research on software code clone detection. The survey explores the reasons for code duplication, its drawbacks, categories of code clones, detection techniques, tool evaluation, visualization methods, and open research problems. This survey used metrics such as portability, precision, recall, scalability, and robustness. It was found that token-based tools often achieve higher recall, and AST/ metrics-based tools achieve higher precision. A broader finding of the study is that no single tool excels in all areas; there exists a trade-off between precision and recall. The paper by Rasha *et al.* [17] presented a systematic literature review of large language models (LLMs) for code completion, analyzing 23 key studies to synthesize advances, challenges, and future directions. The paper offered recommendations on domain-specific LLMs, offering insights for practitioners in this evolving field.

Previous research in code clone detection [14], [15], [16] [10] [17] often focuses on performance metrics or specific LLM evaluations. This paper provides a comprehensive overview, integrating key components to show how LLMs aid in clone detection and refactoring. It uniquely examines the evolving relationship between prompt engineering, tool accessibility, and clone types, highlighting their impact on performance. Rather than providing high-level summaries, it analyzes practical strengths and real-world utility, offering insights for practi-

tioners and researchers. The paper also identifies open problems and suggests directions for future research, fostering innovation in LLM-based clone detection and refactoring.

3. Study Design

3.1. Survey Planning

3.1.1. Identifying the Need for a Systematic Mapping Study

The expanding use of large language models (LLMs) for code quality enhancement and clone detection applications has led to an expanding body of research that assesses these models through various performance metrics, prompt techniques, clone type handling, and practical challenges. There is little systematic review of the specific relationship between LLMs and their applications in code refactoring support and clone detection accuracy.

The main goal of this research is to gather and evaluate recent studies about LLM-based code clone detection systems that focus on their refactoring support features and proposal functions, and identify the need for a systematic mapping study. Therefore, the primary objectives of this study are to:

- Gather existing research on the application of LLMs for code clone.
- Analyze research findings and conclusions to identify benefits and limitations of LLM-based code clone.
- Highlight gaps and challenges to guide future research in this evolving field.

3.1.2. Specifying the Research Questions

During the process of conducting an SLR, it is of paramount importance to pinpoint pertinent research questions that have the potential to provide clear answers. We identified five such research questions (RQs):

- RQ_1. What approaches were considered by the primary studies (PSs) for LLM-driven code clone?
- RQ_2. What datasets and benchmarks have been used to evaluate LLMs for code clone tasks?
- RQ_3. What prompt techniques have been used to perform code clone tasks?
- RQ_4. How accurate are LLM-generated refactoring suggestions?
- RQ_5. What are the key challenges in automating clone detection and refactoring using LLMs?

3.2. Primary Studies Selection

3.2.1. Search Strategy

We searched through major digital libraries which included *IEEE Xplore*[2], *Springer*[3], *Scopus*[4], *Web of Science*[5], *ScienceDirect*[6], *ACM Digital Library*[7], and *Wiley*[8]. Search strings were adapted from prior SLR studies [18] to fit LLM-centric queries.

("software engineering" OR "programming" OR "software development" OR "computer science" OR "computer engineering") AND ("code clone" OR "duplicat" OR "clone" OR "redundant" OR "repetitive") AND ("code") AND ("LLM" OR "large language model" OR "language model") OR "GenAI" OR "Gen AI" OR "Generative AI" OR "GenerativeAI")

TextBox 1: Search string.

3.2.2. Study Selection:

Stage 1: Identification of potentially relevant articles. In this initial phase, we queried well-known digital libraries and used a structured search string listed in textbox 1, tailored to large language models (LLM)-related terminology and code clone topics. The search string was applied to the title, abstract, and keyword fields, yielding a total of 1547 publications. No venue restrictions were imposed to ensure broad coverage across disciplines.

Stage 2: Removal of duplicates. The results from different digital libraries were consolidated through dataset merging, followed by deduplication. The analysis excluded non-peer-reviewed documents. The initial pool decreased to **1479** unique studies after this process.

Stage 3: Exclusion of articles based on title and abstract. We used titles and abstracts to exclude studies irrelevant to our research. The first stage of filtering involved checking whether the articles explicitly mentioned the use of LLMs in code analysis, refactoring, or clone detection. When a determination cannot be reached solely based on the title and abstracts, the studies are promoted to the next stage. This refinement reduced the candidate pool to **68** articles.

Stage 4: Exclusion of articles based on full text. We conducted a full-text screening on the remaining **68** articles and eliminated articles that lacked methodological or empirical evidence about LLMs used for code quality

[2] `https://ieeexplore.ieee.org`
[3] `https://link.springer.com`
[4] `https://www.scopus.com`
[5] `https://www.webofknowledge.com`
[6] `https://www.sciencedirect.com`
[7] `https://dl.acm.org`
[8] `https://onlinelibrary.wiley.com`

enhancement or clone analysis. The final selection of primary studies reached **17** after this stage.

Stage 5: Snowballing. The remaining candidate studies received a full-text review after authors excluded studies based on their titles and abstracts. The authors conducted a manual full-text review of each remaining article to assess its relevance to research questions about LLMs in code clone detection or refactoring.

During the process, each paper was evaluated based on the criteria of whether it proposed or evaluated a method that uses LLMs (e.g., CodeBERT, ChatGPT) for code clone detection or automated refactoring, whether the study provided either quantitative (e.g., precision, recall, F1-score) or qualitative (e.g., human evaluation, prompt design discussion) analysis of the approach, whether the study provided enough technical and empirical information to support classification based on dataset selection, prompt strategy, LLM architecture, and code clone type.

The evaluation process excluded papers that failed to meet the standards because they only mentioned LLMs without any implementation or empirical validation. The evaluation process at this stage excluded both duplicate studies and low-effort opinion pieces. The full-text review stage identified 28 primary studies, which were rigorously screened to address our research questions. The studies provided the foundation for data extraction and analysis in the subsequent review phase. Table 2 illustrates the literature survey process.

3.3. Study Quality Assessment

The full-text review phase included a qualitative assessment through manual inspection to evaluate the relevance and methodological soundness of selected primary studies. Similar to [13], to assess the quality of PSs, we followed the guidelines proposed in previous studies [19–21]. These questions are used to evaluate the quality of each PS across three dimensions: objective, method, and coverage. We assessed each study based on whether each study (i) directly addressed the use of large language models (LLMs) such as ChatGPT in the context of code clone detection or related tasks such as clone classification, and refactoring; (ii) described a defined approach, method, or experiment involving LLMs; and (iii) presented informative insights or results relevant to at least one of our research questions. These questions are implicitly used in the filtering process described above. If a PS meets these quality criteria, we believe it contains valuable information for the SLR. The answer to each of these questions is either "Yes", "Partially", or "No".

3.4. Data Extraction, Categorization, and Analysis

We conducted a full-text review to identify primary studies and then manually extracted data from each selected study using a structured spreadsheet. It lists each primary study and includes fields that correspond to our research questions. The spreadsheet contains multiple fields, which include paper title,

venue, publication year, author list, large language models (LLMs) used, datasets employed, languages tested, prompt strategies, final prompt examples, and evaluation results.

We categorized each study according to the dimensions that emerged during data inspection. The LLM strategies received classification into zero-shot, few-shot, and chain-of-thought prompting categories. The datasets received two types of tags: classification labels (e.g., BigCloneBench, XLCoST) and language-coverage status (mono-lingual versus cross-lingual). The structured tagging system enabled us to study patterns in model usage such as GPT-4, CodeBERT, LLaMA3 and clone types such as Type 1–4 and evaluation outcomes accuracy improvements, mixed results, prompt sensitivity.

To begin the data extraction process, the 1547 candidate papers, narrowed down through prior selection, were evenly divided among the four authors. The authors read their assigned papers to determine whether each study aligned with the research focus on LLMs and code clone detection. Another author performed a thorough verification of each paper's information to ensure accuracy, consistency, and completeness. Corrections or clarifications were made directly in the shared spreadsheet. In cases where disagreements occurred, such as differing interpretations of whether a paper used LLMs for clone detection or disagree on any information entered in the column, we will discuss it in the group meetings. Decisions were finalized only after reaching consensus among the authors, ensuring consistent and reliable classification across the entire dataset.

A holistic review of the extracted data was conducted to identify patterns that appeared across studies. We then identified similar approaches and prompt strategies while noting the commonly used datasets and evaluation methods and the different LLM applications in clone detection tasks. The thematic trends were traced without any subjective interpretation. By organizing the findings along shared characteristics such as LLM type (e.g., GPT, LLaMA, BERT), prompting method, and evaluation design, we ensured a consistent foundation for answering each of the five research questions.

3.5. Final Primary Studies Selection

After completing the full-text review and informal quality assessment to determine the final set of primary studies for our literature review, the main venues for these relevant PSs are presented in Table 1. The review included only studies that specifically focused on large language models for code clone detection and provided enough technical information or experimental findings.

We selected 28 primary studies for data extraction and analysis. The selected studies appeared in peer-reviewed conferences and journals across different venues. The research studies made direct and significant contributions to at least one of the research questions, which this review outlines. The included papers used LLMs such as GPT-4, CodeBERT, and LLaMA3 and zero-shot/few-shot and chain-of-thought as prompting strategies to detect clones in different programming languages and datasets. The selected studies provide the empirical basis for the results and discussion presented in the following sections.

Table 1: Publication venues.

Venue	Publication Venue	PSs
Journal	Journal of Systems & Software	[22] [23] [24]
Conference	International Conference on Code Quality	[25]
Conference	IEEE International Conference on Software Maintenance and Evolution	[26–28] [29]
Workshop	IEEE International Workshop on Software Clones	[30]
Conference	IEEE / ACM International Conference on Program Comprehension	[31, 32]
Conference	IEEE / ACM International Conference on Automated Software Engineering	[33, 34] [35]
Journal	Empirical Software Engineering	[36]
Conference	International Conference on Software Engineering Research, Management and Applications	[37]
Conference	International Conference on the Foundations of Software Engineering	[38]
Conference	Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics	[39]
Journal	IEEE Access	[40] [41]
Conference	CoRR	[42]
Journal	IEEE Access	[43]
Conference	International Conference on Ubiquitous Computing and Intelligent Information Systems	[44]
Conference	Big Data and Cognitive Computing	[45]
Conference	IEEE Transactions on Software Engineering	[46]
Journal	Proceedings of the ACM on Software Engineering	[47]
Journal	Journal of King Abdulaziz University: Computing and Information Technology Sciences	[48]

Table 2: Literature search process.

Item	Count
Scopus	62
Springer	899
IEEE Xplore	15
ACM	6
Wiley	5
ScienceDirect	524
WebOfScience	4
Google Scholar	32
Stage 1: Identify potentially relevant articles	1,547
Stage 2: Remove duplicates	1,479
Stage 3: Exclude articles (Title & Abstract)	68
Stage 4: Exclude Articles (Full text)	17
Stage 5: Snowballing	11
Final Set of Articles	28

4. Experimental Results

4.1. RQ_1: What approaches were considered by the primary studies (PSs) for LLM-driven code clone?

Motivation. Understanding the study's approaches is crucial to interpreting the results and their potential applications. As we seek a better understanding of the applications of large language models (LLMs) in detecting code clones and code refactoring tasks, it is important to recognize the limitations of the study's results given the approach taken. Understanding how a result was reached can provide meaningful context and help identify the most suitable approach in a given context. Furthermore, this enables us to compare and contrast results across studies based on their approaches, informing more informed observations about the use of LLMs in refactoring.

Approach. To understand the approaches taken by the primary studies for LLM-driven code clone detection, we record some key information from each study in Table 3:

- *Focus:* Indicates the main scope of the paper.
- *Clone Types:* Indicates which type(s) of code clones the paper focused on.
- *Refactoring Types:* Indicates which types of refactoring were used in the study.

- *Tools:* Lists all tools the study used.
- *LLMs:* Indicates which Large Language Model(s) the study focused on.
- *Language:* Indicates the programming languages in which the code to be refactored was written.

Results. We discuss results based on clone types, refactoring types, tools, LLMs, and languages.

Clone Types. The most common type of code clone studied was Type-4 clones, which are used in 89% of the studies [22–36, 41, 42, 47–49], followed by Type-3 clones, used in 57% of the studies [22, 24, 25, 27–29, 31, 36, 40, 43–49]. Although Type-1 and Type-2 clones are the easiest to detect, they only appeared in 50% of the studies [22, 24, 25, 27, 29, 31, 36, 40, 43–45, 47–49]. Only 4% of the studies [49] subdivided clone types and specified WT3/T4, MT3, ST3, VST3, and 8% did not discuss code clone types [38, 39].

Refactoring Types. Most of the primary studies either do not discuss code refactoring or do not specify refactoring types, accounting for 79% of the studies [22, 23, 25–29, 31–36, 38–40, 44–49]. The most common types of refactoring are semantic refactoring [43] and functional equivalence [43], [16], each of which is used in 8% % of studies. Other mentioned types of refactoring include renaming, reordering, and restructuring, and only appear in 4% of studies [43].

Tools. The most commonly used tools across the studies included traditional clone detection tools and code representation models. NiCad, Oreo and SourcererCC were among the most frequent, appearing in approximately 11% of studies [16, 17, 46]. CodeBERT and GraphCodeBERT were used in approximately 50% of the articles [22, 26, 27, 29, 34, 36, 39, 41–46, 49]. Newer models such as UniXcoder and CodeT5 appeared in around 14% of the studies, typically alongside other transformer-based models. Other tools and metrics, such as FACER-CD, CLCDSA, and text-embedding-ada-002, and evaluation metrics such as F1 score and cyclomatic complexity, were used in roughly 25% of the papers [26–28, 31, 33, 36, 41]. Approximately 21% of the studies also employed techniques for data visualization (e.g., t-SNE), statistical analysis (e.g., TF-IDF, VSM, Word2Vec) or graph-based analysis (e.g., AST, PDG, GMN) [15, 15].

LLMs. The most commonly used LLMs across the studies were variants of CodeBERT, which appeared in approximately 36% of the studies [15, 17, 42, 43]. ChatGPT models, including GPT-3, GPT-3.5, and GPT-4, were used in about 61% of the studies [16, 17]. Encoder-decoder models such as GraphCodeBERT and PLBART were used in around 22% of the papers. Multilingual or instruction-tuned models such as LLaMA, StarCoder, and Falcon were used in approximately 36% of the works. About 4% incorporated specialized models such as SantaCoder, WizardCoder, or DeepSeek-Coder. UniXcoder appeared in approximately 14% of the studies. In general, CodeBERT and ChatGPT variants were the dominant LLMs.

Language. Java was the programming language most frequently used among the primary studies and was used in 83% of the studies [15–17, 23–25, 28–

30, 35, 36, 38, 40–43, 50, 51]. It is followed by Python, which was used in 50% of studies [17, 23, 25, 28, 36, 43, 45, 46, 51].

Summary for RQ_1. *The 28 primary studies demonstrated that researchers primarily used LLMs to detect complex semantic clones through Type-4 (89% of studies) and Type-3 (57%) clone detection methods. The studies focused on clone detection but failed to specify refactoring types in 79% of their cases, which indicates a disconnect between detection and practical refactoring. The programming language Java led the field with 83% usage, while Python followed at 50%, but multilingual code analysis remained underdeveloped. The research employed CodeBERT and ChatGPT (3.5/4) models as their primary tools. The current approaches focus on semantic detection through a limited set of well-known LLMs, yet they demonstrate weak capabilities in refactoring strategies and language diversity.*

4.2. RQ_2: What datasets and benchmarks have been used to evaluate LLMs for refaccode clone tasks?

Motivation. Determining the datasets and benchmarks used is essential when looking at the reproducibility of studies. As the use of large language models (LLMs) in code clone detection increases, it is important to examine their differences across benchmarks, datasets, and programming languages. Without identifying these differences, it is difficult to compare data and determine what is correct. Studies in this current landscape use different dataset sizes, types, and programming languages. Many rely on crowdsourced data, while others use standardized benchmarks. Table 4 allows for comparisons between these studies, allowing us to look into their similarities and differences before we look at their results.

Approach. To better understand the quality, workflow, and design of the experiments in each of the primary studies, we report in Table 4 the following three characteristics:

- *Source:* Indicates where the dataset or benchmark originated from.
- *Dataset:* Indicates the type and quantity of the data within each dataset or benchmark.
- *Language:* Indicates the programming language(s) represented in the dataset or benchmark.

Table 3: Related work in LLM-driven code clone detection (RQ_1).

Study	Year	Focus	Clone Types	Refactoring Types	Tools	LLMs	Language
Rabbani *et al.* [26]	2022		Type-4	Not Mentioned	FACER-CD, CodeBERT	CodeBERT	Java
Roy *et al.* [10]	2023		Type 4	Semantic Refactoring	NiCad, Oreo	GPT-3	Java, C, C#, Python
Paul*a et al.* [31]	2023		Type-1, Type-2, Type-3, Type-4	None	Graph matching network (GMN), CLCDSA, C4, F1 score, scoML	Transcoder	Java, Python
Li *et al.* [14]	2023		Type-4	Not Mentioned	ZCI, CodeBERT, t-SNE visualization, MAP metric	CodeBERT	C++, Java, Python, C#, Javascript, PHP, C, Ruby
Alam *et al.* [28]	2023		Type-3, Type-4	Not Mentioned	GPT-3, SemanticCloneBench, NiCad, CloneCognition, SourcererCC, Oreo, CLCDSA	GPT-3	Java, C, C++, C#, Python
Zhang & Saber [25]	2024		Type-1, Type-2, Type-3, Type-4	Not Mentioned	ChatGPT 3.5, ChatGPT 4	ChatGPT 3.5, ChatGPT 4	Java, python, C, C#
Khajezade *et al.* [32]	2024		Type-4	None	F1 Score, Cyclomatic Complexity(CC), Acceptance rate, Precision, Recall	ChatGPT 3.5 turbo	Java, Ruby
Khajezade *et al.* [36]	2024		Type-1, Type-2, Type-3, Type-4	None	CodeBERT, GraphCodeBERT, ClonCSE, Contrastive-CoT	CodeBERT, GraphCodeBERT, Contrastic-CoT	Java
Inoue & Higo [37]	2024		Type-4	functional equivalence without syntax reuse	FEMP Dataset (Functionally Equivalent Method Pairs)	GPT-3.5-Turbo, Llama2-Chat-7B, CodeLlama-7B-Instruct	Java
Xie *et al.* [18]	2024		None	Not Mentioned	LLMzipCrazy, GPTZoo	ChatGPT,Claude ,Gemini , LLM zoos from GPT Store, BERT	None
Sorokin *et al.* [39]	2024		None	Not Mentioned	GraphCodeBERT,DeepSeek-Coder-1.3B,CodeBERT,SynCoBERT,RoBERTa-base,ConCode , CodeGen, CodeGPT, StarCoder, Phi-1	DeepSeek-Coder-1.3B,CodeGen, CodeGPT, StarCoder, Phi-1	JavaScript,Java,Python,Go,Ruby
Gupta & Goyal [40]	2024		Type-1, Type-2, Type-3, Type-4	None	LLM: AST, VSM (Vector Space Modelling), LSI (Latent Semantic Indexing), Word2Vec (CBOW and Skip-Gram), FastText, WordNet	ChatGPT (GPT-3.5-turbo)	Java
Mounsuna *et al.* [33]	2024		Type-4	Not Mentioned	text-embedding-ada-002	Falcon-7B-Instruct, LLAMA2-Chat-7B, Starchat-β, GPT-3.5-Turbo	Java, C, Python, etc.
Mounsuna *et al.* [42]	2025		Type-4	Functional Equivalence	ChatGPT-3.5-Turbo, Falcon, StarChat, LLaMA2, GraphCodeBERT, UniXcoder, Support Vector Machines (SVMs) with OpenAI embeddings	GPT-3.5-Turbo, Falcon, StarChat, LLaMA2	Java paired with 10 other languages (e.g., Python, C++, JavaScript, Go, etc.)
Almastadi *et al.* [43]	2025		Type-1, Type-2, Type-3, Type-4	functional equivalence, renaming, reordering, instructuning, semantic-preserving	ChatGPT3.5, GPT-4, CodeBERT, GraphCodeBERT, UniXcoder, CodeT5	ChatGPT-3.5, ChatGPT-4, CodeBERB, GraphCodeBERB, UniXcoder, CodeT5, Starcoder	Java, Python, C++, etc...
Bibekshar [44]	2025		Type-1, Type-2, Type-3, Type-4	None	AST,PTG, LLMs, token and text based technique, machine learning model	CodeBERT,ChatGPT,LLaMA	Java, C, C++
Zhang & Saber [45]	2025		Type-1, Type-2, Type-3, Type-4	Not Mentioned	GPT-3.5, GPT-4,CodeBERT, Cosbo/text-distinct-001	GPT-3.5 and GPT-4	Java, python, C, C#
Zhang & Saber [49]	2025		T1,T2,MT3,T4,ST3,VST3,WT3/T4	Not Mentioned	LLaMA3-8b-Instruct, LLaMA3-70b-Instruct,BigCloneBench, GPTCloneBench,QLoRA, ZeRO	LLaMA3-8B-Instruct, LLaMA3-70B-Instruct, ChatGPT3, CodeBERT	Java
Nashaat *et al.* [22]	2025		Type-1, Type-2, Type-3, Type-4	Not Mentioned	ChromXformer,CodeBERT,GraphCodeBERT,UniXcoder,BigCloneBench	GraphCodeBERT,CodeBERT,UniXcoder	Java
Paul*a et al.* [27]	2025		Type-1, Type-2, Type-3, Type-4	None	CLCMiner, LECCA, AST-based Neural Network(ASTNN), Graph Matching Network (GMN)	CodeBERT, C4, CLCDSA	Java, Python
Lopez *et al.* [46]	2025		Type-3	None	SOURCERERCC, BLEU score, Mean Reciprocal Rank (MRR), LoRA, Prefix-tuning, Layer freezing, Full fine-tuning	CodeBERT, GraphCodeBERT, UNIXCODER, CODET5	Java, Python
Mounsuna *et al.* [47]	2025		Type-1, Type-2, Type-3, Type-4	Not Mentioned	Pre-trained embedding model baseline	GPT-3.5-Turbo, Falcon-7B-Instruct, StarChat-β, LLAMA 2-Chat-7B, StarCoder2-15B-Instruct, plus embedding model text-embedding-3-large	XLCoST, CodeNet
Qian & AlOmar [24]	2025		Type-1, Type-2, Type-3, Type-4 (definition appears in prompt text)	Extract Method refactoring	Baseline refactoring tools JDeodorant and AntCopyPaster	GPT-4o, GPT-5, Claude Sonnet 4, Gemini 2.5, Llama 4	Java
Engel *et al.* [23]	2025		Type-4	None	SLACC (Type-4/semantic clone detector)	GPT-3.5, GPT-4, CodeLlama	C++, Java, Python, Java
Li *et al.* [41]	2025		Type-4	"Code revision" transformations applied to clone instances	Not Specifically Mentioned	CodeBERT, C4, CLCDSA	Java
Alam *et al.* [29]	2025		Type-1, Type-2, Type-3, Type-4	None	Simian, PMD, CPD, Deckard, Clone Digger, CCCD, CloneWorks, Stone Detector, ASTNN, CodeBERT	GPT-3.5, GPTCloneBench	Java, Python, JavaScript, C, C++, C#
Kitsios *et al.* [35]	2025		Type-4	None	Compares task-specific semantic clone detectors vs generative LLMs	llama-3.3-70b-versatile, gpt-4o-2024-08-06, deepseek-r1-distill-qwen	Java, C
Almastadi *et al.* [48]	2025		Type-1, Type-2, Type-3, Type-4	mentions automated refactoring as part of the framework	HICDA hybrid architecture leveraging instruction-tuned LLMs	GPT-4	Not Specifically Mentioned

Results. Our results provide the scope of sources, datasets, and languages used throughout our 28 studies.

Sources. The most common sources were open-source repositories on GitHub and SourceForge, used in 75% of studies [10], [49], [32], [37], [50], [40], [43], [51], [22], [52], [46]. Crowdsourced platforms such as AtCoder, Google CodeJam, and Stack Overflow contributed to datasets of 21% of studies [31], [34], [27], [50], [33], [42]. Datasets generated with LLMs of ChatGPT-3 accounted for 18% of studies [28], [10], [51], [27]. These were used to supplement existing benchmarks with synthetic examples.

Datasets. BigCloneBench was the most widely used benchmark, appearing in 32% of the studies [10], [49], [43], [25], [51], [22], [52], [27], [46]. In addition, 44% of studies [34], [10], [53], [50], [40], [33], [42], [51], [27], [46] used two or more datasets in their evaluations. Dataset sizes varied significantly, ranging from as few as 1,000 code pairs (CLCDSA [31]) to over 66,000 programs (GPTCloneBench [28]). Larger benchmarks such as CSNCC (50,868 programs) [34] provided substantial data for training and evaluation.

Languages. Java was the most represented language, included in 83% of studies [10], [54], [32], [37], [50], [43], [40], [51], [22], [55], [52], [56], [46], [28], [57], [25], [53], [31], [27], [33], [42]. Python followed at 50%, while C and C++ were present in approximately 57% of studies. Other languages, including C#, JavaScript, PHP, and Ruby, appeared in only 36% of studies.

Table 4: Dataset in LLM-driven code clone detection (RQ_2).

Study	Source	Dataset	Language
Rabbani *et al.* [26]	Open-source Java methods curated and human-annotated	SemanticCloneBench: 1,000 Java Method Pairs, Human Annotated Clone Pairs, Positive Examples Only	Java
Roy *et al.* [30]	Open-source Java methods curated and human-annotated, and GPT-generated clone extension	SemanticCloneBench, GPTCloneBench: GPT-3 created 80,664 semantic clone pairs, 23,364 cross language pairs from 15,000 semantic and 12,000 cross language clones	Java, C, C#, Python
Pinku *et al.* [31] '	Competitive programming submissions from AtCoder and Google CodeJam	CLCDSA: 1,095 Java / 1,028 Python Problems, Avg. 55 / 19 Lines	Java, Python
Li *et al.* [34]	Open-source code from GeeksForGeeks, CodeSearchNet, AtCoder, and Google CodeJam	CSNCC: 50K Programs in Java, Python, Ruby; includes CodeJam, AtCoder, BigCloneBench, POJ-104, XLCost, and CSNCC Benchmarks	C++, Java, Python, C#, Javascript, PHP, C, and Ruby
Alam *et al.* [28]	Open-source code from SemanticCloneBench, SourcererCC, Oreo, and CLCDSA curated and extended with GPT	GPTCloneBench: 37,149 True Semantic Clones, 19,288 False Semantic Clones, 20,770 Cross-Language Clones	Java, C, C#, Python
Zhang & Saber [25]	GitHub repositories (IJaDataset, SourcererCC), SemanticCloneBench, GPT-3 (OpenAI API)	BigCloneBench, GPTCloneBench: 300 Datasets Each	Java, Python, C, C#
Khajezade *et al.* [32]	GitHub repositories + Online Coding Platform	CodeNet: Mono-lingual and Cross-lingual Code Clone Detection (CCD)	Java, Ruby
Khajezade *et al.* [36]	GitHub repositories + Online Coding Platform	CodeNet: mono-lingual and cross-lingual code clone detection (CCD)	Java, C++, Ruby
Inoue & Higo [37]	GitHub repositories (IJADataset)	FEMPDataset: Java methods only with 1755 training pairs, 220 validation pairs, 219 test pairs	Java
Xie *et al.* [38]	GPT Store, FlowGPT, Poe, Coze	LLMappCrazy, GPTZoo: 785,129 LLM apps collected	Python
Sorokin *et al.* [39]	Github repositories (IJaDataset, SourcererCC), Multi-lingual Code Retreval	CodeSearchNet, XCD Dataset: 110 problems, 55,000 code snippets	Python, Java, C#, C++, C
Gupta & Goyal [40]	open-source Java repositories with human validation	SemanticCloneBench: 1000 Semantic Clone Pairs	Java, C, Python
Moumoula *et al.* [33]	OGitHub and competitive programming repositories	XLCoST, CodeNet: Not Specified	Java,C,Python, and other total of eleven languages
Moumoula *et al.* [42]	Online coding platforms, GeeksforGeeks programming tutorials	XLCoST: 45,000 - cross-lingual code pairs across 11 languages	Java,C,Python, and other total of eleven languages
Almatrafi *et al.* [43]	Open-source code from GitHub repositories	BigCloneBench: 2,000 samples	Java, C++, Ruby
Bhuiyan [44]	Open-source code from GitHub repositories (IJaDataset, SourcererCC)	BigCloneBench: 8 million labeled clone pairs derived from 25,000 systems	Java, C, C++
Zhang & Saber [45]	GitHub repositories (IJaDataset, SourcererCC), SemanticCloneBench, GPT-3 (OpenAI API)	GPTCloneBench: 1000 clone pairs	Java, Python, C, C#
Zhang & Saber [49]	Open-source code from GitHub repositories	BigCloneBench 6.4M clone pairs; GPTCloneBench: 37K true semantic clones	Java
Nashaat *et al.* [22]	Open-source code from GitHub repositories	BigCloneBench: 6.4M clone pairs	Java
Pinku *et al.* [27]	IJaDataset 2.0 (SourceForge, Google Code), Stack Overflow (crowdsourced code snippets), GPT-3 (OpenAI API)	BigCloneBench: 6.4M clone pairs; SemanticCloneBench: 1,000 Java method pairs; GPTCloneBench: 37,149 true semantic clones	Java, Python
López *et al.* [46]	GitHub open-source repositories, IJaDataset 2.0 (SourceForge, Google Code)	CodeSearchNet: 6.4M snippets (2M docs), 6 languages; CodeTrans: 12K Java/C# pairs; TLC: 87K Java methods; BigCloneBench: 6.4M clone pairs	Python, Java
Moumoula *et al.* [47]	GeeksforGeeks programming tutorials (XLCoST) + competitive programming platforms (CodeNet from AIZU Online Judge + AtCoder)	XLCoST,CodeNet	C, C++, C#, Java, JavaScript, PHP, Python
Qian & AlOmar [24]	GitHub open-source repositories, IJaDataset 2.0 (SourceForge, Google Code)	JFreeChart 1.0.10 (clone groups from JDeodorant reports); plus a JHotDraw case study with manually introduced clone instances	Java
Eagal *et al.* [23]	LeetCode problem descriptions + solutions used as the basis for clone generation	A set of easy LeetCode tasks (listed with LOC per language) and 20 generated snippets per task	Java, C++, Python
Li *et al.* [41]	CodeSearchNet + their constructed syntactic clone benchmark	CodeSearchNet (they state using Python and Java from CodeSearchNet) + a syntactic clone dataset (10M training pairs, 2.5M validation pairs, 20K test pairs)	Python, Java
Alam *et al.* [29]	Human-authored clone benchmarks and AI-era benchmark comparisons	Human-authored clone benchmarks and AI-era benchmark comparisons	Java, Python, C, C#
Kitsios *et al.* [35]	Standard semantic-clone benchmarks	BCBs (BigCloneBench subset), BCBs', SemanticCloneBench, OJ	Java, C
Almatrafi *et al.* [48]	Not specified in the paper's evaluation description	four balanced datasets representing Type-1 to Type-4 clones	Python, Java

Summary for RQ$_2$. *Studies heavily relied on open-source datasets and benchmarks. Java and Python dominated the representation of programming languages. There was limited multilingual and dataset diversity.*

4.3. RQ$_3$: What prompt techniques have been used to perform code clone tasks?

Motivation. In tasks involving large language models (LLMs), prompting often plays a crucial role in shaping the model's responses. There exist countless prompting techniques that can be employed to communicate more efficiently with an LLM and thus improve the accuracy of its response. Specific to certain tasks, and the primary studies employ a wide range of prompting techniques are better suited for certain tasks. Thus, we want to know which of these prompting techniques are being used for code clone detection to understand how effective they are. Understanding the best prompting techniques when it comes to code clone detection and refactoring can greatly improve model performance in these areas.

Approach. We recorded information regarding prompting techniques in Table 5 in order to get a better understanding of the prompt techniques used by the primary studies for clone detection and refactoring tasks.

- *Prompt Technique:* Indicates which prompting technique(s) the study used in performing clone-detection and refactoring tasks.
- *Final Prompt:* The final prompt used in the study, if provided.

Results. There was a wide range of methods used in prompting. Below are the seven categories we have encountered, listed and explained.

- Zero-shot [24, 32, 36, 42, 43, 47, 49, 55]: Provides minimal context and asks the model directly to determine if two code snippets are clones.
- Few-shot [24, 25, 28, 30, 36, 43, 45]: Usually provides definitions of code clone types 1-4 alongside the code snippets and asks the model directly to determine whether or not they are clones.
- Chain-of-thought [42, 43, 54]: Provides a step-by-step process of reasoning to improve the LLM's decision making in code clone detection.
- Contrastive learning [35, 36, 44]: No example prompts were given, though this technique usually provides multiple inputs and asks the model to compare them.

Table 6 presents information on the best prompts used across different papers, along with their explanations. Although not all studies clearly identified the best prompt, some patterns emerge across studies. Some studies (e.g., [24, 25]) used a few-shot technique to produce more stable and clear results, compared to a one-shot technique with only one example. We would suggest

that "few-shot" is most useful in those cases where it helps clarify "task expectation" or reduces variability, but "few-shot" will rarely be needed if the task has been framed as explicitly as possible. Many studies (e.g., [25, 32, 33, 37, 42, 45]) used prompts that forced LLMs to produce a strict answer, most times yes or no. The main purpose of this limitation across research has been to provide a way to increase consistency and automation of evaluation, as unconstrained outputs are often more descriptive and/or vague, making them difficult to evaluate. As shown in Table 6, the most consistent improvement comes from reframing the definition of the task (i.e., asking if two examples solve the same problem/are functionally equivalent), rather than simply varying the number of shots alone, as it will reduce the ambiguity surrounding what "clone" means. In addition, in cross-lingual settings, some studies e.g., [33, 42, 47]) suggest that one should not use the word "clone" in the prompt, as it might confuse LLMs, instead mentioning functional equivalence. Some research highlights repeatability as key, often sidestepping fixed templates altogether [30], suggesting that even with different prompt phrasing, similar results can still be produced. Taken collectively, the data about comparing prompts indicate that, in general, prompts will produce better results (be more effective) if they clearly outline what tasks the model is to perform, avoid using ambiguous terms or words that could be interpreted differently based upon the language used, and strictly restrict the type of response the model is allowed to provide.

Table 5: Prompt technique used in LLM-driven code clone detection (RQ_3).

Study	Prompt Technique	Final Prompt
Rabbani *et al.* [26]	None	None
Roy *et al.* [30]	Few-shot	Not given
Pinku *et al.* [31]	None	None
Li *et al.* [34]	None	None
Alam *et al.* [28]	Few-shot	Give me 10 distinctive implementations for the following code
Zhang & Saber [25]	Few-shot, zero shot	Task instruction to determine if code snippets are clone. Clone type definitions (Type-1 to Type-4). Prompt ends with are these two code snippets clone, answer yes or no. No explanations asked or included
Khajezade *et al.* [32]	Prompt-Based Zero-shot Learning	code1, code2, Do code 1 and code 2 solve identical problems with the same inputs and outputs? Answer with yes or no and no explanation.
Khajezade *et al.* [36]	Zero-shot, Few-shot, Contrastive Prompting	Determine if the following two code snippets are clones
Inoue & Higo [37]	Simple binary prompting	Not given, followed a yes/no binary classification format on a pair of code snippets
Xie *et al.* [38]	None	None
Sorokin *et al.* [39]	None	None
Gupta & Goyal [40]	None	None
Moumoula *et al.* [33]	Used eight prompts designed to elicit a "yes/no" or a similarity score	Not given
	and designed a specific prompt focused on “overall structure and logic”	
	to help guide models	
Moumoula *et al.* [42]	Zero-shot prompting and Chain-of-thought (CoT) prompting	Not mentioned but uses prompts with different level of complexities in its instructions
Almatrafi *et al.* [43]	Prompt-based detection with zero-shot and few-shot methods, CoT	No single prompt
Bbhaskar [44]	Contrastive learning with code pairs, semantic similarity prompts	Not given
Zhang & Saber [45]	Few-shot	Prompt includes clone type definitions. Example prompt ends with: “Are these two code snippets clones? Answer yes or no.”
Zhang & Saber [49]	Zero-shot prompting with custom tags	[INST] Please analyze the following two code snippets and determine if they are code clones. Respond with only ###RESULT###YES for clones or ###RESULT###NO if not. Provide no other output. Code snippet 1: $code_1 Codesnippet2$:code_2 [/INST]
Nashaat *et al.* [22]	None	None
Pinku *et al.* [27]	None	None
López *et al.* [46]	None	None
Moumoula *et al.* [47]	zero-shot and chain-of-thought	8 prompt variants–Simple,Similar-line CoT, Reasoning CoT, Integrate CoT,Separate-code CoT,Separate-explanations CoT,Code-similarity CoT,Improved simple prompt
Qian & AlOmar [24]	zero/one/few-shot prompting for classification	System instruction sets the LLM role, provides Type-1–Type-4 definitions, then includes method info + the entire Java file, and asks the model to identify whether clones exist and return the clone type.
Eagal *et al.* [23]	Prompted clone generation; shows temperature tradeoff	None
Li *et al.* [41]	Explicitly provides a revision prompt template	Uses explicit generation prompts such as: generate a Java semantic code clone of the given code; generate a syntactically different semantic clone; and generate a Java semantic clone from a provided C++ or Python snippet.
Alam *et al.* [29]	None	None
Kitsios *et al.* [35]	Contrastive In-Context Learning	provide contrastive examples and then ask the model to decide for a new pair: determine whether they are clones and provide an explanation.
Almatrafi *et al.* [48]	None	None

Table 6: Best prompt technique used in LLM-driven code clone detection (RQ_3).

Study	Best Prompt	Explanation
Rabbani *et al.* [26]	None	None
Roy *et al.* [30]	Did not mention exact prompt but rather used few-shot prompting and gave textual instructions + a representative input	If any other people try to replicate the research with different prompts, they will get similar results if they follow proper prompt engineering techniques
Pinku *et al.* [31]	None	None
Li *et al.* [34]		
Alam *et al.* [28]	None	None
Zhang & Saber [25]	Specifically used few-shot prompting	Mentioned their initial experiment with one-shot prompt brings inconsistent results
Khajezade *et al.* [32]	Provide 2 code and ask do code 1 and code 2 solve identical problems with the same inputs and outputs? answer with yes or no and no explanation	They found that directly asking "are these clones?" is ineffective, so they redesigned the prompt to ask whether the two snippets solve the same problem and have the same inputs/outputs, and they force a strict Yes/No output
Khajezade *et al.* [36]	None	None
Inoue & Higo [37]	None	None
Xie *et al.* [38]	None	None
Sorokin *et al.* [39]	None	None
Gupta & Goyal [40]	None	None
Moumoula *et al.* [33]	Consider the overall structure and logic of the following two codes and determine if the two code snippets perform a similar task. Respond with 'yes' if the two codes perform similar tasks or 'no' otherwise	LLMs struggle to grasp how "code clones" work when dealing with a cross-lingual setting. Some systems actually fail; saying similar bits in different tongues shouldn't be considered duplicates. Offering clear definitions helps a lot - removing the term "clone" makes a big difference in outcome.
Moumoula *et al.* [42]	Consider the overall structure and logic of the following two codes and determine if the two code snippets perform a similar task. Respond with 'yes' ... or 'no' ...	LLMs often don't understand what "clone" means in a cross-lingual setting, and the word "clone" can be misleading. Since responses were sometimes right, it became clear LLMs claimed to rely on overall structure and logic. Because of this, prompts were reshaped - "clone" dropped entirely, replaced by instructions that actively steer thinking toward that exact idea.
Almatrafi *et al.* [43]	They sets the system with a role("Code Clone Detection Task") and frame the task as a short chat, and the user supplies two code snippets and asks whether they are clones, and the assistant responds with a simple labeled decision	Adding labeled examples / diverse cases helps the model distinguish clone types
Bbhaskar [44]	None	None
Zhang & Saber [45]	Used a one zero-shot instruction prompt that says: "Please analyze the following two code snippets and determine if they are code clones," then requires the model to output only ###RESULT###YES (clone) or ###RESULT###NO (not a clone), with the two snippets inserted as $code_1 and $code_2.	Used this format because when they asked for plain "yes/no," the model sometimes produced extra text
Zhang & Saber [49]	Used a one zero-shot instruction prompt that says: "Please analyze the following two code snippets and determine if they are code clones," then requires the model to output only ###RESULT###YES (clone) or ###RESULT###NO (not a clone), with the two snippets inserted as $code_1 and $code_2.	Used this format because when they asked for plain "yes/no," the model sometimes produced extra text
Nashaat *et al.* [22]	None	None
Pinku *et al.* [27]	None	None
López *et al.* [46]	None	None
Moumoula *et al.* [47]	Consider the overall structure and logic of the following two codes and determine if the two code snippets perform a similar task. Respond with 'yes' if the two codes perform similar tasks or 'no' otherwise.	Lots of LLMs get confused by "clone" when working across languages. Even if answers were right, folks behind the study noticed models claimed decisions came from structure and reasoning, not just wording. So changes were made - the task now has new wording, skipping that tricky term altogether.
Qian & AlOmar [24]	Used zero-shot: Told LLMs to act as Java developer, then clear definitions outline from Type-1 to Type-4 copies, and provided the method details + full Java file; The output ends by showcasing both generated codes alongside a label stating their matching category	After testing with zero shot, few shot, and one shot, they found one-shot/few-shot did not improve results and would increase computational cost and time.
Eagal *et al.* [23]	Generate a Java semantic code clone of the code below: reference code", where reference code is a LeetCode solution	Minimal prompts were applied just to probe baseline clone-generation ability and to minimize wording effects across models. Modifications to the syntactically different prompt were made sparingly to test Type-IV clones under strict comparison.
Li *et al.* [41]	Rewrite the following Python code to achieve the same functionality, but use different syntax. Keep the logic unchanged while varying variable names, loops, or control structures.	The prompt helps create sentence variations that mean the same but sound different. Doing this repeatedly builds up more examples, showing the system many ways to express identical ideas.
Alam *et al.* [29]	None	None
Kitsios *et al.* [35]	The prompt asks the LLM to decide whether c1,c2 is a clone pair based on a provided definition, and provide an example for clone and non-clone, or a contrastive non-clone example that shares one snippet with the clone example.	One clone + one non-clone is used to keep the prompt short
Almatrafi *et al.* [48]	None	None

Summary for RQ$_3$. *Studies used various prompting techniques to assess LLM performance in code clone detection and refactoring. Few-shot and context-specific prompts were most common, each in 65% of studies. Zero-shot prompting, used in 41%, often served as a baseline or simple method with minimal context. One-shot prompting, which provides a single example, was used in 6%. Chain-of-thought (CoT) prompting, involving step-by-step reasoning, was used in 24%, helping improve decision-making in complex tasks. These findings show growing emphasis on prompt engineering to optimize LLMs for software engineering.*

4.4. RQ$_4$: How accurate are LLM-generated refactoring suggestions?

Table 7: Quantitative, qualitative, and comparative analysis of LLM-driven code clone detection (RQ_4).

Study	Quantitative	Qualitative	Comparative	Correctness
Rahman et al. [26]	SemanticCloneBench	Not specified	FACER-CD, CodeBERT	37,149 true semantic clones, 20,770 cross-language pairs; Benchmark outperformed SemanticCloneBench, BigCloneBench, CLCDSA
Roy et al. [28]	SemanticCloneBench, GPTCloneBench	9 human evaluators	NiCad, Oreo	62.14% accuracy, BLEU = 0.55 (semantic clones); 91.25% accuracy (cross-language); Oreo recall = 0.46
Pinku et al. [31]	CLCDSA	Not specified	C4, mcML-GMN (Graphing Matching Network)	GMN F1: 0.91 → 0.94; CLCDSA F1: 0.66 → 0.69; C4 F1: 0.96 → 0.98; Precision: 0.90, Recall: 0.92; mutation-based augmentation reduced GMN F1 by 3%
Li et al. [34]	CSSCC, BigCloneBench	Not specified	ZCL/CodeBERT, t-SNE visualization, MAP metric	MAP score improvement = +67.12%, +51.37%, +14.83%, +53.01% over SOTA baseline
Akuru et al. [28]	GPTCloneBench	9 human evaluators (6 undergrad students, 3 post-docs)	GPT-3.5(SemanticCloneBench), NiCad, CloneCognition, SourcererCC, Oreo, CLCDSA	37,149 semantic clones, 20,770 cross-language pairs; extensive validation applied
Zhang & Salwe [23]	BigCloneBench, GPTCloneBench	Not Specified	ChatGPT-35, and ChatGPT 40	GPT-4 outperformed GPT-3.5 across clone types; Recall low on human-written Type-4 semantic clones; bias toward LLM-generated clones.
Khajezade et al. [32]	CodeNet	Not Specified	F1 Score, Cyclomatic Complexity(CC), Acceptance rate, Precision, Recall	Baselines: F1: 0.66 (Java-Ruby); precision/recall near 0
Khajezade et al. [36]	CodeNet	Not Specified	CodeBERT, GraphCodeBERT, CloCSE, ContrastiveCuT	Performance dropped 50% (Java) and 20% (C++/Ruby) on unseen problems; MAML boosted F1 by 13-24%; zero-shot remained low for unseen languages/problems
Inoue & Higo [37]	FEMPDataset (Functionally Equivalent Method Pairs)	Not Specified	GPT-3.5-Turbo, Llama2-Chat-7B, CodeLlama-7B-Instruct	LLMs correctly identified functionally equivalent methods; improved accuracy over syntax-based methods
Xie et al. [38]	LLMappCorp, GPTZoo	Not Specified	ChatGPT, Claude, Gemini, LLMappCorp, GPTZoo	Accurate Results, LLmappCorp was effective
Sorokin et al. [39]	CodeSearchNet, XCD	Not Specified	GraphCodeBERT,DeepSeek-Coder-1.3B,CodeBERT,SynCoBERT,RoBERTa-base, CuCode , CodeGen, CodeGPT, SantaCoder, Phi-1	CCT-LLMs: executed all models in multi- and cross-lingual evaluation
Gupta & Goyal [40]	Semantic Clone Bench	Not Specified	LLM, AST, VSM (Vector Space Modeling), LSI (Latent Semantic Indexing), Word2Vec (CBOW and Skip-Gram), FastText, WordNet	Recall 67.81%, Precision 60.06%; best: VSM, ChatGPT; docs improved results; comparable/improved vs prior studies
Mounsamy et al. [16]	XLCoST, CodeNet	Not Specified	Falcon7B-Instruct, LLAMA2-Chat-7B,StarchatB,GPT-35-Turbo,text-embedding-ada002	ChatGPT-3.5 performed well in simple cases but poorly for complex/cross-lingual clones; pre-trained embedding model outperformed LLMs
Mounsamy et al. [42]	XLCoST	Not Specified	ChatGPT-35-Turbo, Falcon, StarChat, LLaMA2, GraphCodeBERT, UniXcoder, Support Vector Machine (SVMs)	CoT prompting improved LLMs in some cases; struggled with semantic reasoning in cross-lingual settings, outperformed LLMs overall
Alnuzaili et al. [43]	BigCloneBench	Not Specified	ChatGPT-35, GPT-4, CodeBERT, GraphCodeBERT, UniXcoder, CodeT5, Starcoder	LLMs strong for syntactic clone detection; struggled with semantic types; embedding models and hybrid techniques often outperformed basic prompting
Bibekar [44]	BigCloneBench	Not Specified	AST, PTG, CodeBERT, ChatGPT, LLaMA	Improved detection accuracy across all clone types, reduced false positives/negatives, processing time by 35%
Zhang & Salwe [45]	GPTCloneBench	Not Specified	GPT-3.5, GPT-4, CodeBERT, Codex/text-davinci-003	Strong on syntactic clones (Type-1,2); weak on semantic clones; more errors on human-written vs LLM-generated code
Zhang & Salwe [46]	BigCloneBench, GPTCloneBench	Not Specified	LLaMA3-8B-Instruct, LLaMA3-70B-Instruct, ChatGPT-4, CodeBERT	LLaMA3 models achieved perfect recall for T1, T2 clones; poor performance on semantic clones (MT3, T4)
Nashaat et al. [22]	BigCloneBench	Not Specified	CloneXformer, CodeBERT, GraphCodeBERT, UniXcoder, BigCloneBench	Recall 0.64, Precision 0.86, F1 0.74; Type-1 precision 0.96, 35% faster than conventional models
Pinku et al. [27]	BigCloneBench, SemanticCloneBench, GPTCloneBench	Not Specified	CLCMiner, LICCA, ASTNN, GMN	Graph-based techniques outperformed baselines with up to 20P higher f1 score
Lopez et al. [46]	CodeSearchNet	Not Specified	SOURCERERCC, BLEU score, Mean Reciprocal Rank (MRR), LoRA, Prefix-tuning, Layer freezing, Full fine-tuning	More frozen layers increased code duplication; inter-dataset code leakage confirmed in LLMs
Mounsamy et al. [47]	XLCoST subset, CodeNet subset	Not Specified	5 LLMs × 8 prompts + 2 baselines + embedding-classifier baseline	F1 up to 0.99 (simple) and 0.75 (hard); embedding classifier F1 0.98 (XLCoST) and 0.74 (CodeNet)
Quan & Abreu [24]	JDeodorant, 300 clones (100 each for Type-1/2/3) detected via JDeodorant	Manual analysis of code clone fragments	JDeodorant, AutoCorpProtor	Type-1 classification perfect; Type-2 accuracy up to 57%, Type-3 up to 9.1%; traditional tools produced 100% correct, compilable refactorings, while best LLMs produced ≈10-11 correct refactorings in their setting.
Ragoi et al. [24]	30 LeetCode problems, generating 25,500 clone candidates	Not Specified	GPT-35, GPT-4, CodeLlama across temperature, clone typing (Type I vs II) + semantic correctness evaluation.	Reported Type 1/II clone distribution by model-temperature and semantic-correctness trends; AST-distance negativity correlation with correctness ($\beta = -0.0020$, $p < 0.001$)
Li et al. [48]	GCJ,BigCloneBench	Not Specified	Compared to NiCad/DECKARD/ASTNN/CodeBERT, GraphCodeBERT/UniXCoder/PLBART/etc.; also compared against LLM prompting baselines.	On BigCloneBench: P=0.97, R=0.96, F1=0.97; Type-4 F1 97.4% vs 95.6%
Akuru et al. [29]	GPTCloneBench, BigCloneBench, SemanticCloneBench	Expert feedback/interviews 5 experts; concerns about equivalence, thresholds, and correctness.	Evaluated 9 CCD tools and detailed preprocessing normalization impacts.	Large F1 gains after preprocessing; in large projects with injected clones, recall reported as 54.5% (Type-2), 28.5% (Type-3), 42.6% (Type-4).
Kistler et al. [35]	BCB, SCB, OJ	Qualitative error/behavior analysis of a CL experiment where performance didn't increase	Re-evaluates 6 SOTA models and compares standard vs contrastive-learning variants.	Cross-functionality setting causes task-specific models F1 drop up to 48% (avg 31%), while LLMs drop up to 5% (avg 3%); contrastive methods improve up to 20% (avg 9%) for task-specific and up to 5% (avg 3%) for LLMs.
Alnuzaili et al. [49]	Dataset names not specified	Not Specified	Compared against BERT and KNN baseline	Reported high scores per clone type

Motivation. The motivation for looking into the accuracy of large language models (LLMs) generated refactoring suggestions is because traditional code clone detection software was used in the past. With LLMs, we can be more efficient, faster, and more effective at large-scale code clone detection and classification. Because of this, our methods must keep pace with current developments, particularly given the rise of LLMs that can solve problems at much higher speeds than we currently can. With an expansion of studies surrounding this topic in the last year, it is important to look into how accurate the LLM-generated refactoring suggestions are.

Approach. To better understand the precision of LLM-generated refactoring suggestions in primary studies, we report in Table 7 the following types of data analysis:

- *Quantitative:* Indicates the dataset used to assess the accuracy of LLM-generated refactoring suggestions.
- *Qualitative:* Indicates the number and type of surveyed participants.
- *Comparative:* Indicates the tools used for comparison.
- *Correctness:* Indicates the results for measuring the accuracy of LLM-generated refactoring suggestions.

We report the correctness metrics of each study as 'Precision', 'Accuracy', 'Recall', or 'F-measure' when explicitly mentioned in the primary studies, and as 'Unknown' if no performance metrics are provided. We mark the qualitative column as 'No' if no qualitative analysis was done. In the comparative column, we record the names of the tools used for comparison, marking 'No' if no comparative analysis was conducted, or 'Unknown' if the tools used for comparison were not named.

Table 8 presents the performance comparison between the traditional detection methods and the LLM-based methods. In Table 8, it is shown how much greater the ΔF_1 values are from the traditional methods (NiCad [1], Deckard [58], and SourcererCC [59]) for studies included. There were significant variations across studies regarding the degree of improvement ΔF_1 achieved by the LLM-based methods. For example, some studies presented values for ΔF_1 of less than 0.01 or even slightly negative (meaning that they did not outperform the traditional methods); whereas, many other studies demonstrated significant improvement over the baseline values for ΔF_1 of greater than 0.20 and one study reported a value of $+0.90$, which represented the greatest difference in performance between the older, non-LLM based clone detection solutions and the newer, LLM-based solutions. Overall, the results suggest that in cases where the traditional clone detectors do not provide acceptable results due to their inability to address more difficult or semantically complicated problems, the use of LLMs may offer more viable options. However, if the traditional method achieves the same level of precision and recall as the LLM method, the benefit of using the LLM solution may be minimal or even counterproductive for the majority of LLM models.

Table 8: Performance comparison between traditional clone detectors and LLM-based approach.

Approach	P	R	F1	Δ F1 vs NiCad	Δ F1 vs Deckard	Δ F1 vs SourcererCC	Accuracy (%)	PSs
NiCad	0.76	0.70	0.73	–	+0.70	–	–	[41]
Deckard	0.93	0.02	0.03	-0.70	–	–	–	[41]
LLM-based (GraphCodeBERT)	**0.93**	**0.94**	**0.93**	**+0.20**	**+0.90**	–	–	[41]
LLM-based (UniXCoder)	**0.92**	**0.93**	**0.925**	**+0.195**	**+0.895**	–	–	[41]
LLM-based (GPT-3.5)	**0.89**	**0.86**	**0.875**	**+0.145**	**+0.845**	–	–	[41]
LLM-based (GPT-4)	**0.93**	**0.94**	**0.94**	**+0.21**	**+0.91**	–	–	[41]
NiCad	0.65	0.83	0.73	–	+0.37	–	–	[22]
Deckard	0.37	0.35	0.36	-0.37	–	–	–	[22]
LLM-based (GraphCodeBERT)	**0.59**	**0.78**	**0.67**	**-0.06**	**+0.31**	–	–	[22]
LLM-based (UniXCoder)	**0.24**	**0.50**	**0.32**	**-0.41**	**-0.04**	–	–	[22]
LLM-based (CloneXformer)	**0.64**	**0.86**	**0.74**	**+0.01**	**+0.38**	–	–	[22]
Deckard	0.02	0.93	0.03	–	–	+0.02	–	[45]
SourcererCC	0.01	0.88	0.01	–	-0.02	–	–	[45]
LLM-based (CodeBERT)	**0.934**	**0.947**	**0.941**	–	**+0.911**	**+0.931**	–	[45]
LLM-based (FA-AST + GMN)	**0.924**	**0.934**	**0.929**	–	**+0.899**	**+0.919**	–	[45]
LLM-based (LLaMA3-70b-instruct)	**0.044**	**0.885**	**0.034**	–	**+0.004**	**+0.024**	–	[45]
LLM-based (ChatGPT-3.5)	**Not Mentioned**	**Not Mentioned**	**0.58**	–	–	–	–	[25]
LLM-based (ChatGPT-4.0)	**Not Mentioned**	**Not Mentioned**	**0.77**	–	–	–	–	[25]
LLM-based (GPT-3.5-Turbo)	**0.69**	**0.84**	**Not Mentioned**	–	–	–	68%	[37]
LLM-based (GPT-4-turbo)	**0.74**	**0.91**	**Not Mentioned**	–	–	–	76%	[37]
Nicad	0.95	0.82	0.88	–	–	-0.02	–	[43]
Oreo	0.91	0.89	0.90	+0.02	–	+0.00	–	[43]
SourcererCC	0.93	0.87	0.90	+0.02	–	–	–	[43]
LLM-based (GPT-3.5 Turbo)	**0.79**	**0.91**	**0.84**	-0.04	–	-0.06	–	[43]
LLM-based (GPT-4)	**0.84**	**0.91**	**0.87**	-0.01	–	-0.03	–	[43]
LLM-based (CodeBert)	**0.88**	**0.85**	**0.86**	-0.02	–	-0.02	–	[43]
LLM-based (GraphCodeBERT)	**0.89**	**0.87**	**0.88**	+0.01	–	-0.01	–	[43]
JDeodorant	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	100%	[24]
AntiCopyPaster	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	100%	[24]
LLM-based (Gemini 2.5 Pro)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	15%	[24]
LLM-based (GPT-3.5)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	Not Mentioned	[24]
LLM-based (GPT-4o)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	70%	[24]
LLM-based (GPT-5)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	75%	[24]
LLM-based (DeepSeek-R1)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	35%	[24]
LLM-based (Claude Sonnet 3.7)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	55%	[24]
LLM-based (Claude Opus 4.1)	Not Mentioned	Not Mentioned	Not Mentioned	–	–	–	50%	[24]
Deckard	0.42	0.17	0.24	–	–	–	–	[29]
ASTNN	0.54	0.31	0.39	–	+0.15	–	–	[29]
LLM-based (CodeBERT)	**0.51**	**0.91**	**0.65**	–	+0.41	–	–	[29]
FACER-CD	Not Mentioned	0.643	Not Mentioned	–	–	–	–	[26]
LLM-based (CodeBERT)	Not Mentioned	0.832	Not Mentioned	–	–	–	–	[24]
CLCDSA	0.47	Not Mentioned	Not Mentioned	–	–	–	–	[28]
SourcererCC	0.82	0.33	Not Mentioned	–	–	–	–	[28]
Oreo	0.49	Not Mentioned	Not Mentioned	–	–	–	–	[28]
LLM-based (Falcon-Instruct-7B)	0.50	0.86	0.63	–	–	–	–	[47]
LLM-based (LLaMA2-Chat-7B)	0.50	1.0	0.67	–	–	–	–	[47]
LLM-based (StarChat-Beta-16B)	0.45	0.30	0.36	–	–	–	–	[47]
LLM-based (Starcoder2-15b-instruct)	1.00	0.01	0.03	–	–	–	–	[47]
LLM-based (GPT-3.5-Turbo)	0.99	0.97	0.98	–	–	–	–	[47]

Results. The following findings describe the quantitative, comparative, and correctness of the studies.

Quantitative. Precision, recall, and F1-score were the quantitative metrics most commonly used, appearing in approximately 43% of studies [31], [28], [54], [36], [32], [49], [51], [25], [27], [46] These metrics primarily assess metric performance and measure how effectively LLMs can detect code clones. In addition, BLEU scores were reported in one study [34] to assess cross-language clone detection performance.

Qualitative. We observe that the majority primary studies lack qualitative analysis. Only a few studies performed the analysis manually or with experts.

Comparative. Comparative analysis was performed in 76% of the studies [10], [31], [34], [28], [49],[36], [32], [37], [53], [50], [43], [25], [51], [22], [56], [27], [46], with models commonly evaluated against baselines of CodeBERT, GraphCodeBERT, SourcererCC, and Oreo. Li *et al.* [34] reported ZC3 achieving 67% MAP improvement over baselines in cross-language clone detection. Pinku *et al.* [27] found graph-based techniques outperforming baselines by up to 20 percentage points in F1-score.

Correctness. Eleven studies reported results supporting the accuracy of

LLM-generated refactoring suggestions in code clone detection tasks [28], [43], [56], [54], [37], [36], [34], [22], [27], [51], [49]. Chen *et al.* [56] reported F1-score improvements of up to 10.81 across refactoring tasks, while Khajezade *et al.* [36] found ChatGPT achieving an F1-score of 0.877, outperforming CodeBERT's 0.663. Alam *et al.* [28] demonstrated 99% functional correctness, validated through unit testing and review by nine human evaluators. Li *et al.* [34] reported their ZC3 model achieving a 67% MAP improvement over baselines in cross-language clone detection, and Pinku *et al.* [27] found graph-based techniques outperforming previous methods by up to 20 percentage points in F1-score.

Summary for RQ$_4$. *Studies evaluated LLM performance using quantitative metrics precision, recall, and F1-score, while qualitative assessments and functional correctness evaluations appeared in only a few cases. Comparative analysis was common, with many studies benchmarking against tools such as CodeBERT and GraphCodeBERT.*

4.5. RQ$_5$: What are the key challenges in automating clone detection and refactoring using LLMs?

Motivation. Using large language models (LLMs) for clone detection offers significant potential to reduce technical debt and improve software maintainability. However, the automation faces key challenges. Primary studies highlight issues with prompt design and inconsistent results across different clone types. These obstacles hinder the reliable integration of LLMs into real development workflows. Recognizing these barriers is essential to guide future research and increase tool adoption.

Approach. The approach we decided to take was to conduct an analysis of 28 primary studies about LLM applications for code clone detection and refactoring to observe the main obstacles. We compared prompt formats, including zero-shot, few-shot, and chain-of-thought (CoT), and assessed their effects on model performance. Additionally, the analysis evaluated prompt strategies between few-shot and zero-shot approaches while examining different clone types (Type-1 to Type-4) accuracy and model generalization across BigCloneBench, GPTCloneBench, XLCoST, and other datasets. The analysis also examined prompt template design and formatting when available.

Results. This section discusses the key challenges encountered when automating code clone using LLMs.

Prompt sensitivity and inconsistency. Many studies such as Dou *et al.* [54], Zhang *et al.* [45], Almatrafi *et al.* [43], mentioned that LLM performance depends heavily on the way prompts are constructed. The results from few-shot prompts with defined clone-type definitions were better, but the absence of standardized prompt templates across studies created variability, which made reproducibility difficult. Some papers (e.g., Dou *et al.* [54], Almatrafi *et al.* [43], Chen *et al.* [56], Zhang *et al.* [45]) reported prompt bias effects, where slight changes in wording produced dramatically different outputs.

Insufficient generalization across programming languages. Many studies (e.g., Moumoula *et al.* [33], Khajezade *et al.* [36], Li *et al.* [34]) also mentioned that clone detection becomes challenging for LLMs when the codebase contains different programming languages, even though the logical operations remain equivalent.Although careful thought has gone into the wording of the cross-language clone detection prompt, it is clear that translation still generates false positives and negatives. This indicates that there is a difference between the semantic meaning in different languages. Current models lack sufficient representational capacity to maintain consistent semantic meaning across programming languages. Their effectiveness declines when working in development environments that use multiple programming languages.

Inter-dataset code duplication. Inter-dataset leakage [46] poses a significant and frequently underestimated issue in assessing the performance of LLMs. This form of leakage is particularly deceptive because it undermines established safeguards designed to separate training and test data. When a model is exposed to previously seen samples during evaluation, it can produce responses that appear remarkably accurate or well-structured, not due to genuine reasoning or generalization, but as a consequence of prior familiarity. In reality, such inflated performance reflects exposure bias rather than authentic understanding, leading authors and practitioners to overestimate the model's true proficiency. This situation is made worse by data-related issues. This information could be compromised by XLCoST's method of releasing project-level data. This could have a particularly negative effect in situations where learning is context-dependent. Alam *et al.* [29] mentioned that due to the lack of clones generated by AI tools and ambiguity in the definition of Type-4 clones in the BigCloneBench, it is insufficient for assessing the current clone detection tools. BigCloneBench lacks clones generated by AI tools. Furthermore, there is an ambiguity in the definition of Type-4 clones in BigCloneBench. This ambiguity exists because the definition of Type-4 clones is that they are the clones of a code snippet that itself is a clone.

Difficulty in detecting Type-4 clones. The primary challenge with identifying Type-4 clones stems from their lack of superficial features that traditional clone detection systems typically rely on. Variable names, loop constructs, and even overall code structure may be completely different across semantically equivalent implementations. This forces large language models (LLMs) to shift focus from surface analysis to deep semantic reasoning. However, most LLMs are trained to excel in pattern recognition rather than in computational equivalence, making it difficult for them to recognize that two dissimilar-looking code snippets might, in fact, be performing the same operation. Moreover, semantic clone detection often demands contextual understanding, domain-specific knowledge, and abstract reasoning. LLMs may struggle when asked to infer logical equivalence across fragments that use divergent programming paradigms, rely on implicit side effects, or involve mathematically identical but structurally distinct algorithms. This limitation makes Type-4 clone detection a particularly hard problem in software engineering.

Operational reliability and practical deployment constraints. Beyond

accuracy, several studies [23],[43] highlighted the issue of the system's operational constraints as a major hurdle in the development of clone detection and refactoring systems powered by large language models. While clones produced by a learned cloning system may be functional, they can be non-executable, in that they cause the program to crash or timeout. Alternatively, they may not behave consistently with the original, so the developer would need to run additional tests. Given a system that uses a large language model to perform semantic inference across the board, it is actually more suitable for the system to reserve its expensive semantic reasoning capabilities for more complex tasks, rather than apply the system's inference uniformly across the board.

Summary for RQ$_5$. *The implementation of LLMs for clone detection and refactoring automation currently encounters several obstacles, including prompt sensitivity and restricted language generalization and Type-4 semantic clone detection challenges. The results of LLMs become highly unpredictable when the prompt wording changes slightly because most research lacks standardized prompt templates, which creates challenges for reproducing and comparing results. The lack of semantic alignment between languages prevents LLMs from performing well in software environments that use multiple languages. The detection of Type-4 clones proves challenging because it needs deep logical reasoning abilities, which most LLMs do not possess.In addition, the difficulty of producing reliable models for a particular domain is a challenge; conversely, LLMs may produce models that are not deployable due to their outputs being either non-executable or unworkable. With intelligent hybrids, it is possible to avoid expensive semantic processing by only using it on more complex cases rather than always using it.*

5. Discussion and Open Issues

Implications of the widespread use of BigCloneBench. Our study reveals that BigCloneBench is the most widely used benchmark in the primary studies evaluated. While its widespread adoption provides a common ground for comparison, the research community must be aware of recently identified limitations in this benchmark that call into question the validity of many published results. First, BigCloneBench is unsuitable for evaluating machine learning approaches targeting Type-4 clones. Krinke and Ragkhitwetsagul [52, 60] found that the ground truth for such clones is unreliable, as a manual analysis of a sample showed an 86% false-positive rate. Since many clone detection tools aim to identify these semantic clones, evaluating them on flawed ground truth may be unreliable. Second, the benchmark's internal structure poses a significant threat to the validity of machine learning experiments. Schäfer *et al.* discovered that functions in BigCloneBench are not intersection-free, so the same code fragments can appear in multiple sections of the benchmark [61]. This may lead

to data leakage between training and evaluation sets, artificially boosting performance metrics. The authors showed that when a more rigorous, functionally based data separation is enforced, a model's F1-score can drop by up to 17%, suggesting that many published results may be overly optimistic. Given these observations, researchers should exercise caution when using BigCloneBench for evaluation and be transparent about their data separation strategies.

The choice of clone detection tool will depend on the type of clones being detected, the number of clones, and the restrictions placed on deploying these clones. The selection of clone detection tools should be based on the clones' expected characteristics rather than solely on how new the clone detection model is. Static clone detectors are typically chosen as the primary clone detector in the case of working within very large, rapidly changing code bases, where fast turnaround times are critical, and determinism is necessary. They are generally considered mature clone detectors and have been used for many years. NiCad was specifically developed to provide a customizable, embeddable, command-line tool that can take input parameters for filtering/normalization and near-miss thresholds that can be configured to generate output in either XML or HTML formats. Deckard has been used to generate empirical results on large-scale, real-world systems with high throughput, utilizing the combination of AST vectorization and efficient clustering. Similarly, token-based detectors such as CCFinder are used when teams need language-flexible, industrial-scale scanning across large codebases. When AI-assisted software development is incorporated into the development process, there will likely be increased variability. Recent research (e.g., Alam *et al.* [29]) indicates that even though the variability may increase, classical clone detectors will still be effective. However, the detection of Type-4 clones will likely remain difficult, and thus, the use of learning-based/LLM methods will most likely be restricted to those candidates that appear to be clearly ambiguous. That is, utilize traditional, efficient clone detectors to detect candidate clones broadly and reserve the "semantic reasoning" provided by LLM/DL based tools for only the candidate clones that appear to be complex or borderline to minimize costs and operational risks associated with non-executable outputs.

Beyond the characteristics of clones, users will have to make a deliberate cost-benefit determination as to which method is preferable, given the trade-off between throughput/operational overhead versus semantic coverage. Because local machine-based detectors are generally less expensive to develop since they require minimal user input, can execute at high scales with minimal resources, and do not require additional development, traditional detectors offer many advantages over LLM-era detectors. Similar to Deckard's goal of achieving high scalability on large, real-world systems (JDK and the Linux kernel), LLM-era detectors can also incur overhead (for example, preprocessing/function extraction) and longer runtime times depending on configuration. For instance, Alam *et al.* [29] has demonstrated that including CodeBERT and ASTNN in the results of their work would be too costly because of "huge time" and the need to process input data through function extraction and preprocessing.

The current evaluation methods do not measure how LLMs such as ChatGPT, CodeBERT, and GPT-4 perform in realistic, long-term development settings. The term "realistic, long-term development settings" here refers to the daily work bases of using LLM tools, encompassing not only the IDE but also code review and repository updates, which are affected by ongoing code integration and changing requirements. Beyond numerical assessment, true evaluation includes encountering actual obstacles that need to be recognized and resolved during the work. The evaluation process needs to assess developer outcomes through performance indicators such as false-positive burden, verification/editing effort, and adoption/revert rates. Selective manual review would also be used by compilation and test suite execution to validate correctness. Most assessments of LLM-based clone detection and refactoring have been limited to using automated metrics and a few relatively small-scale benchmarking efforts (e.g., BigCloneBench and GPTCloneBench). Automated metrics (e.g., precision, recall, and F1) allow for direct comparison of performance; however, automated metrics fail to reflect the performance and usability of LLMs over time within actual, real-world environments. The effectiveness of models like ChatGPT 3.5, GPT-4, and GraphCodeBERT can be influenced significantly by the degree of experience among individual developers, as well as the patterns of collaboration exhibited by development teams. However, the inclusion of contextual variables is typically not included when evaluating these models. Therefore, the results of existing research lack generalizability. Longitudinal empirical research is required that integrates these tools into the actual work flows of software engineers. To achieve this, it will be necessary to observe how developers interact with LLMs during debugging, refactoring, code reviews, etc., and track changes in their confidence, trust, and overall satisfaction with the use of LLMs for clone detection and refactoring. A longitudinal research design will provide the most practical and robust understanding of the potential real-world impacts of LLM-based clone detection and refactoring tools. Thus, the research findings address both RQ1 and RQ5, and demonstrate that while precision and recall metrics continue to be useful, they do not accurately measure the real-world performance and usability of LLMs in clone detection tasks.

The inconsistent application of prompts in prompt engineering creates major reliability problems when comparing LLM performance in clone detection tasks. The studies we focused on that employed GPT-4, LLaMA3 and CodeLlama models revealed that prompt formulation served as the primary factor responsible for clone detection performance variations. The research employed two different approaches to prompt their models: few-shot prompting with defined clone types from Type-1 to Type-4 and zero-shot formats and chain-of-thought reasoning with natural language examples. The failure of numerous studies (e.g., Roy *et al.* [30], Inoue *et al.* [37], Almatrafi *et al.* [43], Bhaskar *et al.* [44], Pinku *et al.* [27], Nashaat *et al.* [22], Krinke *et al.* [52], Xie *et al.* [38], Moumoula *et al.* [47]) to disclose their complete prompt templates created challenges for both experiment replication and fair LLM comparison. The lack of standardized prompts creates challenges for performance

assessment because it is difficult to determine whether enhanced precision results from model capabilities or from improved prompt strategies. Research should make structured prompt libraries available to the public while documenting their prompt construction methods. The lack of transparent prompt design in benchmarking LLMs, including GPTClone, ChatGPT, and Contrastive-CoT, creates reproducibility issues that diminish the validity of research findings. Research outcomes will become more consistent when authors establish shared guidelines for prompt design and make reusable prompt banks available for publication. The inconsistent prompt design and the lack of a standardized prompt structure create obstacles that are addressed in RQ5. The absence of standardized prompts and transparency makes it challenging to conduct fair performance comparisons between LLMs and determine the actual boundaries of LLM-based clone detection systems.

Type-4 Clone detection can be enhanced via logical abstraction and semantic reasoning. As discussed in RQ5, Type-4 clones are among the most challenging categories for clone detection because they lack syntactic similarity and rely on semantic equivalence. LLMs detect clones across a clone pair, which they get as input. These clone-pairs are two instances of code, same language or cross-language, that the LLM is required to check for finding whether they are clones or not. As mentioned in [27] [32], LLMs find it difficult to detect these types of clones in the clone-pairs they receive, because these clone-pairs often perform the same task. Still, their way of implementation is different from one another in the structure in which it implements logic, the data structures used within them, and the sequence in which it performs the task. All of these factors accumulated, make the task of detecting Type-4 clones difficult for the LLMs. Since Type-4 clones have semantic equivalence at their very core, a potential strategy that arises to enhance detection will involve fine-tuning LLMs to capture the fundamental logic of each code in the clone pair and then check for their equivalence. To advance clone detection capabilities, it will be essential for LLMs to possess the ability to deeply engage with and transform the core logic embedded within clone-pairs. This means going beyond superficial pattern matching or structural alignment and instead developing a clear understanding of the underlying computational intent.

Narrow evaluation metrics limit practical insight. 43% of the studies we examined focus on metrics like precision, recall, and F1-score, with very few considering more practical aspects such as code quality or how developers would actually use these tools. While precision and recall are useful for model evaluation, most studies also relied on strict evaluation metrics that only assessed whether the output perfectly matched the benchmark. This binary approach means solutions are either marked as correct or incorrect, with no room for partial matches or outputs that, while not exact, could still be useful to developers in real-world scenarios. None of the studies included human developer analysis of the generated solutions, which is a major gap. A tool can perform well on paper but produce something that is hard to read, or maintain into already existing projects. Without involving developers to assess usability, maintainability, and overall trust in the output, it is hard to know if these tools would

actually save time or introduce new challenges in real-world workflows. For example, an LLM might identify a clone correctly but rewrite it in a way that does not align with project coding standards, making it less helpful in practice.
The limited types of prompting inhibit our ability to determine what works best for this particular task. The studies we looked at almost exclusively used zero-shot and few-shot prompting techniques. Although this provides a strong understanding of the utility of such methods for code clone detection tasks, it offers little insight into the effectiveness of other prompting methods. The only other specific methods mentioned in these studies are chain-of-thought, which appears in 24% of studies, and contrastive learning, which only appears in 8% of studies. Although useful, this does not provide us with much evidence with respect to their utility for this particular task, and completely neglect the multitudes of other potential prompting techniques, such as context-specific, output constraints, or ranking techniques.

6. Threats To Validity

In this section, threats are discussed in the context of four types of validity threats: internal validity, external validity, construct validity, and conclusion validity.
Internal validity. Obtaining a representative set of publications for this mapping study may pose a validity threat due to the search process. To minimize this threat, we followed the literature review guidelines proposed by Kitchenham *et al.* and Wohlin [19, 62]. We used related search terms and the main terms from the research questions to build the search string and identify relevant articles. Furthermore, we followed a study selection process and applied the inclusion and exclusion criteria at each stage.
External validity. The collected papers include a substantial number of academic works that provide a solid foundation for deriving findings potentially valuable to academia. However, we cannot claim that the same approaches are used in industry. Additionally, our findings primarily concern code clones. We cannot generalize our results beyond this subject.
Construct validity. Threats to construct validity involve the appropriateness of the research questions and the categorization scheme used for data collection. To mitigate these threats, the research questions and the categorization schemes were discussed among the authors.
Conclusion validity. Given the subjectivity inherent in assessing PSs, the primary studies were independently reviewed by at least four authors to mitigate bias during data extraction. In cases of disagreement, the researchers discussed them to reach a consensus.

7. Conclusion

In this paper, we map and review the body of knowledge on large language models (LLM)-based code clone approaches. We systematically reviewed 28 papers and classified them. Our findings show that **(i)** The majority LLM-driven

code clone detection studies focus on Type-4 clones (85 %), followed by Type-3 (57%), while Type-1 and Type-2 appear in just 50%. Clone subtypes such as WT3/T4 and MT3 are mentioned in only 4%, and 8% of studies do not specify clone types. Refactoring is largely underexplored: 89% of papers omit it or generalize it. Semantic refactoring and functional equivalence are each used in 8%, with only 4% mentioning renaming, reordering, or restructuring. Java dominates the programming language landscape with 83% usage, while Python is used in 50%; **(ii)** Studies utilized primarily datasets from open-source repositories like GitHub and SourceForge. The most utilized benchmark was BigCloneBench, and nearly half of the studies utilized more than a single dataset for assessment. Java was the most favored language, followed by Python and C/C++. Other languages like C#, JavaScript, PHP, and Ruby were also utilized in some cases.; **(iii)** Seven categories of prompt techniques were found to be used to perform refactoring tasks. These were: zero-shot, one-shot, few-shot, chain-of-thought, context-specific, output constraints and ranking; **(iv)** LLMs for code clone detection were largely evaluated using precision, recall, and F1-score, with 72% of studies adopting these metrics. Comparative analysis in 76% of studies highlighted strong baselines like CodeBERT and GraphCodeBERT, while ZC3 showed a 67% MAP boost and graph-based methods improved F1-score by up to 20 points. Refactoring precision was supported in 11 studies—ChatGPT achieved a 0.877 F1-score, and some models reached 99% functional correctness through expert validation; and **(v)** The key challenges in automating clone detection and refactoring using LLMs are: prompt sensitivity and inconsistency, insufficient generalization across programming languages, inter-dataset code duplication, and Type-4 clone detection enhancement.

Declaration of generative AI and AI-assisted technologies in the writing process. During the preparation of this work, the author used the ChatGPT Web interface to improve the language and readability. After using this tool, the authors reviewed and edited the content as needed and took full responsibility for the publication's content.

References

[1] J. R. Cordy, C. K. Roy, The nicad clone detector, in: 2011 IEEE 19th international conference on program comprehension, IEEE, 2011, pp. 219–220.

[2] T. Kamiya, S. Kusumoto, K. Inoue, Ccfinder: A multilinguistic token-based code clone detection system for large scale source code, IEEE transactions on software engineering 28 (7) (2002) 654–670.

[3] N. Tsantalis, T. Chaikalis, A. Chatzigeorgiou, Ten years of jdeodorant: Lessons learned from the hunt for smells, in: 2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER), IEEE, 2018, pp. 4–14.

[4] N. Tsantalis, A. Chatzigeorgiou, Identification of extract method refactoring opportunities for the decomposition of methods, Journal of Systems and Software 84 (10) (2011) 1757–1782.

[5] E. A. AlOmar, A. Ivanov, Z. Kurbatova, Y. Golubev, M. W. Mkaouer, A. Ouni, T. Bryksin, L. Nguyen, A. Kini, A. Thakur, Anticopypaster: extracting code duplicates as soon as they are introduced in the ide, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–4.

[6] E. A. AlOmar, B. Knobloch, T. Kain, C. Kalish, M. W. Mkaouer, A. Ouni, Anticopypaster 2.0: Whitebox just-in-time code duplicates extraction, in: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, 2024, pp. 84–88.

[7] E. A. AlOmar, J. Ashkenas, R. Feliciano, M. Angelakos, D. Haralamppopoulos, X. Qian, M. W. Mkaouer, A. Ouni, Anticopypaster 3.0: Just-in-time clone refactoring, ACM Transactions on Software Engineering and Methodology.

[8] E. A. AlOmar, A. Ivanov, Z. Kurbatova, Y. Golubev, M. W. Mkaouer, A. Ouni, T. Bryksin, L. Nguyen, A. Kini, A. Thakur, Just-in-time code duplicates extraction, Information and Software Technology 158 (2023) 107169.

[9] S. Martinez, L. Xu, M. Elnaggar, E. A. Alomar, Software refactoring research with large language models: A systematic literature review, Journal of Systems and Software (2025) 112762.

[10] C. K. Roy, J. R. Cordy, A survey on software clone detection research, in: Technical Report No. 2007-541, School of Computing, Queen's University, Queen's University at Kingston, Ontario, Canada, 2007, pp. 1–90.

[11] M. Fowler, Refactoring: improving the design of existing code, Addison-Wesley Professional, 2018.

[12] E. A. AlOmar, M. W. Mkaouer, C. Newman, A. Ouni, On preserving the behavior in software refactoring: A systematic mapping study, Information and Software Technology 140 (2021) 106675.

[13] E. A. AlOmar, M. W. Mkaouer, A. Ouni, Behind the intent of extract method refactoring: A systematic literature review, IEEE Transactions on Software Engineering 50 (4) (2024) 668–694.

[14] M. Lei, H. Li, J. Li, N. Aundhkar, D.-K. Kim, Deep learning application on code clone detection: A review of current knowledge, Journal of Systems and Software 184 (2022) 111141.

[15] A. Sheneamer, J. Kalita, A survey of software clone detection techniques, International Journal of Computer Applications 137 (10) (2016) 1–21.

[16] D. Rattan, R. Bhatia, M. Singh, Software clone detection: A systematic review, Information and Software Technology 55 (7) (2013) 1165–1199.

[17] C. C. Rasha Ahmad Husein, Hala Aburajouh, Large language models for code completion: A systematic literature review, in: Computer Standards & Interfaces, Volume 92, Qatar University, Doha, Qatar, 2025.

[18] N. Raihan, M. L. Siddiq, J. C. Santos, M. Zampieri, Large language models in computer science education: A systematic literature review, in: Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1, 2025, pp. 938–944.

[19] B. Kitchenham, S. Charters, et al., Guidelines for performing systematic literature reviews in software engineering (2007).

[20] S. Li, H. Zhang, Z. Jia, C. Zhong, C. Zhang, Z. Shan, J. Shen, M. A. Babar, Understanding and addressing quality attributes of microservices architecture: A systematic literature review, Information and software technology 131 (2021) 106449.

[21] T. Dybå, T. Dingsøyr, Empirical studies of agile software development: A systematic review, Information and software technology 50 (9-10) (2008) 833–859.

[22] M. Nashaat, R. Amin, A. H. Eid, R. F. Abdel-Kader, An enhanced transformer-based framework for interpretable code clone detection, Journal of Systems and Software 222 (2025) 112347.

[23] A. Eagal, K. T. Stolee, J.-P. Ore, Analyzing the dependability of large language models for code clone generation, Journal of Systems and Software (2025) 112548.

[24] X. Qian, E. A. Alomar, Can large language models identify and refactor code clones? an empirical study, Journal of Systems and Software (2025) 112717.

[25] Z. Zhang, T. Saber, Assessing the code clone detection capability of large language models, in: 2024 4th International Conference on Code Quality (ICCQ), IEEE, 2024, pp. 75–83.

[26] S. M. Rabbani, N. A. Gulzar, S. Arshad, S. Abid, S. Shamail, A comparative analysis of clone detection techniques on semanticclonebench, in: 2022 IEEE 16th International Workshop on Software Clones (IWSC), IEEE, 2022, pp. 16–22.

[27] S. N. Pinku, D. Mondal, C. K. Roy, On the use of deep learning models for semantic clone detection, in: 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2024, pp. 512–524.

[28] A. I. Alam, P. R. Roy, F. Al-Omari, C. K. Roy, B. Roy, K. A. Schneider, Gptclonebench: A comprehensive benchmark of semantic clones and cross-language clones using gpt-3 model and semanticclonebench, in: 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2023, pp. 1–13.

[29] A. I. Alam, P. Roy, F. Al-omari, C. Roy, B. Roy, K. Schneider, Are classical clone detectors good enough for the ai era?, 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME) (2025) 1–13.

[30] P. R. Roy, A. I. Alam, F. Al-Omari, B. Roy, C. K. Roy, K. A. Schneider, Unveiling the potential of large language models in generating semantic and cross-language clones, in: 2023 IEEE 17th International Workshop on Software Clones (IWSC), IEEE, 2023, pp. 22–28.

[31] S. N. Pinku, D. Mondal, C. K. Roy, Pathways to leverage transcompiler based data augmentation for cross-language clone detection, in: 2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC), IEEE, 2023, pp. 169–180.

[32] M. Khajezade, J. J. Wu, F. H. Fard, G. Rodríguez-Pérez, M. S. Shehata, Investigating the efficacy of large language models for code clone detection, in: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, 2024, pp. 161–165.

[33] M. B. Moumoula, A. K. Kabore, J. Klein, T. F. Bissyande, Cross-lingual code clone detection: When llms fail short against embedding-based classifier, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 2474–2475.

[34] J. Li, C. Tao, Z. Jin, F. Liu, G. Li, Zc 3: Zero-shot cross-language code clone detection, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2023, pp. 875–887.

[35] K. Kitsios, F. Sovrano, E. T. Barr, A. Bacchelli, Detecting semantic clones of unseen functionality, International Conference on Automated Software Engineering.

[36] M. Khajezade, F. H. Fard, M. S. Shehata, Evaluating few-shot and contrastive learning methods for code clone detection, Empirical Software Engineering 29 (6) (2024) 163.

[37] R. Inoue, Y. Higo, Improving accuracy of llm-based code clone detection u sing functionally equivalent methods, in: 2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications (SERA), IEEE, 2024, pp. 24–27.

[38] Y. Xie, X. Hou, Y. Zhao, K. Chen, H. Wang, Llm app squatting and cloning (2025) 64–74.

[39] N. Sorokin, D. Abulkhanov, S. Nikolenko, V. Malykh, Cct-code: Cross-consistency training for multilingual clone detection and code search (2025) 178–185.

[40] A. Gupta, R. Goyal, A generative ai-driven method-level semantic clone detection based on the structural and semantical comparison of methods, IEEE Access.

[41] C. Li, J. Konpang, A. Sirikham, Y. Wang, Nuanced code clone detection through llm-based code revision and ast graph modeling, IEEE Access 13 (2025) 191024–191036.

[42] M. B. Moumoula, A. K. Kaboré, J. Klein, T. F. Bissyandé, Large language models for cross-language code clone detection, CoRR.

[43] A. A. Almatrafi, F. A. Eassa, S. A. Sharaf, Code clone detection techniques based on large language models, IEEE Access.

[44] P. V. Bhaskar, et al., A comprehensive analysis of unified approaches for revealing code clone detection, in: 2024 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS), IEEE, 2024, pp. 946–953.

[45] Z. Zhang, T. Saber, Exploring the boundaries between llm code clone detection and code similarity assessment on human and ai-generated code, Big Data and Cognitive Computing 9 (2) (2025) 41.

[46] J. A. H. López, B. Chen, M. Saad, T. Sharma, D. Varró, On inter-dataset code duplication and data leakage in large language models, IEEE Transactions on Software Engineering.

[47] M. B. Moumoula, A. K. Kaboré, J. Klein, T. F. Bissyandé, The struggles of llms in cross-lingual code clone detection, Proceedings of the ACM on Software Engineering 2 (FSE) (2025) 1023–1045.

[48] A. A Almatrafi, F. A Eassa, S. A Sharaf, Hybrid intelligent architecture for context-driven code clone detection, Journal of King Abdulaziz University: Computing and Information Technology Sciences 14 (2) (2025) 44–50.

[49] Z. Zhang, T. Saber, Assessing code clone detection capabilities of large language models on human and ai-generated code: Zero-shot and fine-tuning approaches, Available at SSRN 4979508.

[50] A. Shirafuji, Y. Oda, J. Suzuki, M. Morishita, Y. Watanobe, Refactoring programs using large language models with few-shot examples, in: 2023 30th Asia-Pacific Software Engineering Conference (APSEC), IEEE, 2023, pp. 151–160.

[51] B. Zhang, P. Liang, Q. Feng, Y. Fu, Z. Li, Copilot-in-the-loop: Fixing code smells in copilot-generated python code using copilot, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 2230–2234.

[52] J. Krinke, C. Ragkhitwetsagul, How the misuse of a dataset harmed semantic clone detection, arXiv preprint arXiv:2505.04311.

[53] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, L. Zhang, ismell: Assembling llms with expert toolsets for code smell detection and refactoring, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1345–1357.

[54] S. Dou, J. Shan, H. Jia, W. Deng, Z. Xi, W. He, Y. Wu, T. Gui, Y. Liu, X. Huang, Towards understanding the capability of large language models on code clone detection: A survey, arXiv preprint arXiv:2308.01191.

[55] M. Liu, J. Li, Y. Wang, X. Du, Z. Ou, Q. Chen, B. An, Z. Wei, Y. Xu, F. Zou, et al., Code copycat conundrum: Demystifying repetition in llm-based code generation, arXiv preprint arXiv:2504.12608.

[56] Q. Chen, L. Jiang, I. Ahmed, From bias to improved prompts: A case study of bias mitigation of clone detection models, arXiv preprint arXiv:2505.05679.

[57] F. Leone, Semantic clone detection without type limitations.

[58] L. Jiang, G. Misherghi, Z. Su, S. Glondu, Deckard: Scalable and accurate tree-based detection of code clones, in: 29th International Conference on Software Engineering (ICSE'07), IEEE, 2007, pp. 96–105.

[59] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, C. V. Lopes, Sourcerercc: Scaling code clone detection to big-code, in: Proceedings of the 38th international conference on software engineering, 2016, pp. 1157–1168.

[60] J. Krinke, C. Ragkhitwetsagul, Bigclonebench considered harmful for machine learning, in: 2022 IEEE 16th International Workshop on Software Clones (IWSC), IEEE, 2022, pp. 1–7.

[61] A. Schäfer, W. Amme, T. S. Heinze, Experiments on code clone detection and machine learning, in: 2022 IEEE 16th International Workshop on Software Clones (IWSC), IEEE, 2022, pp. 46–52.

[62] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th international conference on evaluation and assessment in software engineering, 2014, pp. 1–10.